



GEMINI

AGENT ADK

COURSE SLIDES · WSQ

Develop Multi AI Agent Applications with Gemini Agent ADK

WSQ Course Code: TGS-2024042961

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Version v1.1 · 11 August 2026

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



COURSE ADMINISTRATION

Welcome & Housekeeping

Digital Attendance (Mandatory)

1

Three times a day

Take the AM, PM and Assessment digital attendance — mandatory for every WSQ-funded course.

2

Trainer shows the QR

The trainer or administrator displays the digital attendance QR code from the SSG portal.

3

Scan and submit

Scan the QR code with your mobile phone camera and submit your attendance.

4

75% minimum

A minimum of 75% attendance is required to be eligible for assessment and funding.

About the Trainer



Your Trainer

General Trainer template —
to be completed by the trainer

NAME

TITLE / DESIGNATION

QUALIFICATIONS

AREAS OF EXPERTISE

TRAINING & INDUSTRY EXPERIENCE

CONTACT

About the Trainer



Dr Alfred Ang

Principal Trainer
Tertiary Infotech Academy Pte Ltd

ROLE

Principal Trainer, Tertiary Infotech Academy Pte Ltd

EXPERTISE

Generative AI, Large Language Models, agentic AI systems and applied machine learning.

DELIVERS

WSQ courses on AI, LLM application development, prompt engineering and data science.

FOUNDER

Founder and lead instructor at Tertiary Infotech / Tertiary Courses.

Let's Know Each Other

- Your name, organisation and role.
- Your experience with Python, APIs and Generative AI tools (if any).
- What you would like to automate or build with AI agents after this course.

Ground Rules

1

Set your mobile phone to silent mode.

2

Participate actively — no question is too small.

3

Mutual respect: agree to disagree.

4

One conversation at a time.

5

Be punctual; return from breaks on time.

6

75% attendance is required.

Download Your Course Material

1

1 · Go to the LMS portal

Open <https://lms-tms.tertiaryinfotech.com> in your browser.

2

2 · Log in

Sign in with the account e-mail you used to register for this course.

3

3 · Open this course

Select 'Develop Multi AI Agent Applications with Gemini Agent ADK'.

4

4 · Download

Download the Trainer Slides, Learner Guide and Lesson Plan (PDF).

5

5 · Get the labs

Clone the lab repository from GitHub — the link is on the course page.

6

6 · Keep them open

You may use these materials during the open-book assessment.

Skills Framework Alignment

1

TSC Title

Artificial Intelligence Application

2

TSC Code

AER-TEM-4026-1.1

3

A1 / A2

Analyse algorithms in AI applications; correlate algorithm design with efficiency.

4

A3 / A4

Identify and compare the strengths and limitations of AI applications.

5

A5

Assess the feasibility of AI applications for engineering processes.

6

A6

Assess the improvements AI applications bring to engineering processes.

SCHEDULE

Lesson Plan — 2 Days, 8 hours/day

Day 1

- Day 1 — Agentic AI foundations with Gemini ADK — first agents, tools, sessions and multi-agent handoff
 - Digital Attendance (AM) · Introductions · Learning Outcomes
 - Topic 1: Overview of Agentic AI in Gemini ADK (Labs 1–4)
 - Lunch Break · Digital Attendance (PM)
 - Topic 2: Build A Multi Agent App with Gemini ADK begins (Labs 5–8)

Day 2

- Day 2 — Multi-agent systems, agentic RAG and shipping an agent app with Streamlit
 - Topic 2 continues (Labs 9–12)
 - Topic 3: Build Agentic AI RAG in Gemini ADK (Labs 13–15)
 - Topic 4: Agentic AI App with Streamlit (Labs 16–18)
 - TRAQOM Survey · Digital Attendance (Assessment)
 - Final Assessment (WA + PP)

Learning Outcomes

1

LO1 · Analyse LLM applications

Analyze the range of LLM applications using Generative AI and identify their industrial use cases.

2

LO2 · Establish Gemini designs

Establish Google Gemini GAI designs and assess improvements on engineering processes.

3

LO3 · Develop LLM applications

Develop LLM applications with the Agent Development Kit and assess their feasibility.

4

LO4 · Evaluate RAG

Evaluate the performance effectiveness of Retrieval Augmented Generation.

Course Outline

1

Topic 1 — Overview of Agentic AI in Gemini ADK

LLM & agentic AI foundations · the Gemini model family · ADK architecture · your first agent · tools.

2

Topic 2 — Build A Multi Agent App with Gemini ADK

Sessions & state · handoff · sub-agents · workflow agents · guardrails · structured output · MCP.

3

Topic 3 — Build Agentic AI RAG in Gemini ADK

The RAG pipeline · loading & splitting · embeddings · vector stores · retrieval · evaluation.

4

Topic 4 — Agentic AI App with Gemini ADK and Streamlit

YAML agents · Streamlit chat UI · async Runner · deployment and feasibility · capstone.

BEFORE YOU START

Briefing for Assessment

1

Do · Clear your desk

Place phones and other materials under the table or on the floor.

2

Don't · No recording

No photos or recording of assessment scripts.

3

Don't · No discussion

Work individually — no discussion during the assessment.

4

Do · Black or blue pen

Use a black or blue pen for hard-copy assessments.

5

Don't · No correction fluid

No liquid paper or correction tape may be used.

6

Do · Stop on time

Scripts are collected when time is up.

Assessment

1

Written Assessment (WA)

Short-Answer Questions (SAQ) · 1 hour · open book.
Tests the underpinning knowledge taught in the slides.

2

Practical Performance (PP)

Hands-on Gemini ADK agent build tasks · 1 hour · open book. Tests the ability you built in the labs.

3

Open book

You may use the course slides, the Learner Guide and approved materials only.

4

Eligibility

A minimum of 75% attendance is required to be eligible for assessment and funding.

5

Result

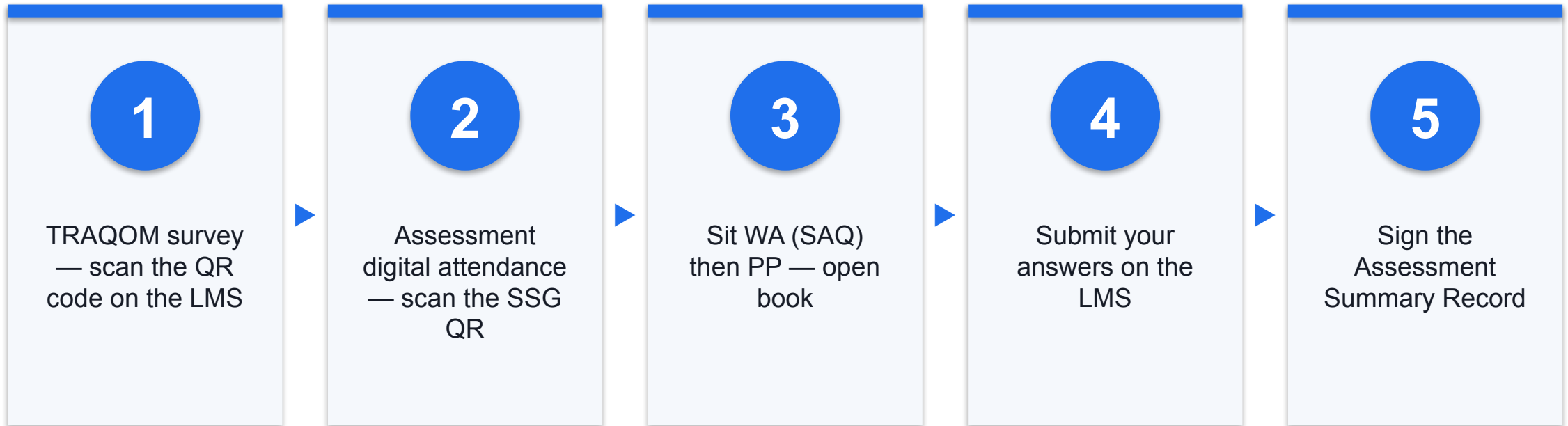
You are assessed as Competent (C) or Not Yet Competent (NYC) on each instrument.

6

Appeals

An appeal process is available if you wish to contest an assessment outcome.

Assessment Flow



Criteria for Funding

1

Attendance

A minimum attendance rate of 75%, based on the SSG Digital Attendance record.

2

Assessment

Complete both assessment components and be assessed as 'Competent'.

3

Digital attendance

Scan the SSG QR code for AM, PM and Assessment on every training day.

4

TRAQOM survey

Complete the mandatory TRAQOM course feedback survey on the LMS.

Set Up Before We Start

1

Google account

Sign in to Google AI Studio at <https://aistudio.google.com> to create a free API key.

2

Python 3.13

Install Python 3.13 and the uv package manager for the lab environment.

3

Lab repository

Clone the course labs from GitHub — the link is on your LMS course page.

4

Editor

Use VS Code or any editor you are comfortable with for editing agent.py files.

5

Optional keys

OpenWeather and Tavily free API keys are used by the tool-calling labs.

6

ADK docs

Bookmark the official ADK documentation at <https://google.github.io/adk-docs/>.



CORE CONCEPTS

From Large Language Models to AI Agents

What is a Large Language Model?

1

A model of language

An LLM such as Gemini, GPT-4 or Claude is trained to understand and generate human language.

2

Why 'large'

Large in both the volume of training data and the model's complexity — hundreds of billions of parameters.

3

Autoregressive

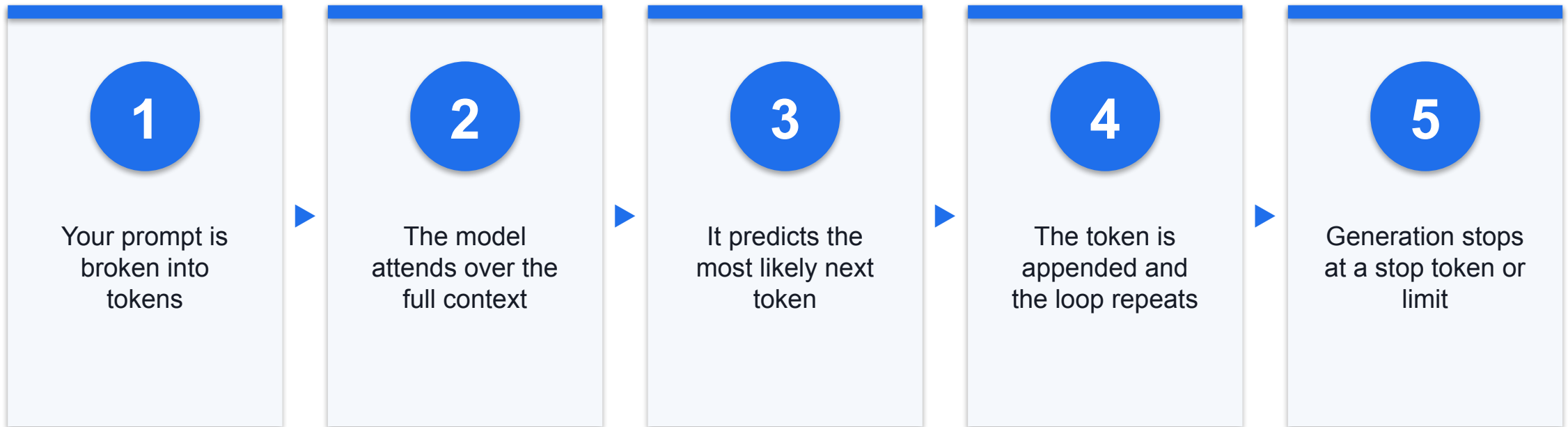
It predicts the next token, one token at a time, conditioned on everything written so far.

4

Trained then inferred

Training builds the model from a huge corpus; inference is the prompt-and-response you use every day.

How an LLM Produces an Answer



Opportunities of LLM Applications

1

Content creation and writing assistance

2

Customer support and chatbots

3

Language translation and localisation

4

Educational and tutoring tools

5

Business intelligence and analytics

6

Coding and software development

7

Legal and compliance assistance

8

Healthcare and clinical documentation support

9

Enhanced search and knowledge discovery

10

Accessibility for persons with disabilities

Industry Use Cases of Generative AI

Finance & Legal

- Automated financial reporting
- Risk assessment and analysis
- Fraud detection
- Contract review and drafting
- Regulatory compliance monitoring

Marketing & eCommerce

- Content and copy generation
- Personalised recommendations
- Customer sentiment analysis
- SEO and campaign optimisation
- Product description generation

Education & Healthcare

- Personalised learning assistance
- Automated grading and feedback
- Clinical documentation
- Medical imaging support
- Virtual health assistants

Popular Large Language Models

1

Google Gemini

Natively multimodal; Ultra, Pro, Flash and Nano variants — the model family used in this course.

2

OpenAI GPT

Widely adopted general-purpose family; reachable from ADK through the LiteLlm wrapper.

3

Anthropic Claude

Strong long-context reasoning and writing; also reachable through LiteLlm.

4

Open models

Meta Llama and Mistral can be self-hosted or run locally through Ollama for data-residency needs.

WHY AGENTS

An LLM answers. An agent acts.

Agentic AI adds reasoning, memory, tool use and autonomy on top of a language model — so it can complete tasks, not just produce text.

What Makes a System 'Agentic'?

1

Reasoning

The model plans a sequence of steps instead of answering in one shot.

2

Tool use

It calls functions and APIs to fetch live data or take real actions.

3

Memory

Session state lets it carry context across many turns of a conversation.

4

Autonomy

It decides which tool or which specialist agent handles the request.

5

Collaboration

Multiple specialised agents divide the work and hand off between themselves.

6

Guardrails

Callbacks and policies constrain what the agent is allowed to do.

Single Agent vs Multi-Agent

Simple

- Single agent
 - One instruction, one set of tools
 - Simple to build and to debug
 - Instruction grows unwieldy as scope widens
 - Best for a narrow, well-defined job

Scalable

- Multi-agent
 - Each specialist has its own instruction and tools
 - Scales to broad problem domains
 - Routing and handoff must be designed carefully
 - Best for varied requests across several sub-domains

What is Gemini?

1

Built by Google DeepMind

Gemini is Google's flagship model family, built from the ground up for multimodality.

2

Natively multimodal

It reasons across text, images, video, audio and code in a single model.

3

Long context

Very large context windows allow whole documents, codebases and videos in one prompt.

4

Tool-ready

Native function calling makes Gemini a natural fit for agentic applications.

The Gemini Model Family

Pro

- The mid-tier, balanced option
- Strong reasoning, broad multimodal skill
- Optimised for performance, cost and latency
- The default choice for most agents

Flash

- Lightweight and fast
- Optimised for speed and efficiency
- Low cost per call at high volume
- Used in most labs in this course

Ultra & Nano

- Ultra: the most capable, for highly complex tasks
- Nano: the most efficient, runs on-device
- Nano targets mobile and offline scenarios
- Choose by task complexity and where it runs

Model Parameters You Can Tune

1

Temperature

Controls randomness. Low values give focused, repeatable answers; high values give creative, varied ones.

2

Top-K

Restricts sampling to the K most probable next tokens. Top-K of 1 always picks the single most likely token.

3

Top-P

Samples from the smallest set of tokens whose probabilities sum to P — a dynamic alternative to Top-K.

4

Max output tokens

Caps the length of the response, which directly caps cost.

5

Stop sequences

Strings that end generation immediately when produced — useful for structured formats.

6

Safety settings

Thresholds that filter harmful content across several harm categories.

What is the Agent Development Kit (ADK)?

1

Open-source framework

Google's Python framework for building, evaluating and deploying AI agents.

2

Model-agnostic

Runs on Gemini natively, and on OpenAI, Anthropic or local models through LiteLLm.

3

Batteries included

Sessions, tools, callbacks, multi-agent orchestration and evaluation are all built in.

4

Developer UI

adk web gives you a local browser IDE that shows every event, tool call and trace.

Anatomy of an ADK Agent

1

model

Which LLM powers the agent — for example gemini-2.0-flash.

2

name

A unique identifier used in traces and in agent-to-agent transfers.

3

description

What this agent is for. A parent agent reads this to decide whether to hand off to it.

4

instruction

The system prompt: the agent's role, rules, tone and boundaries.

5

tools

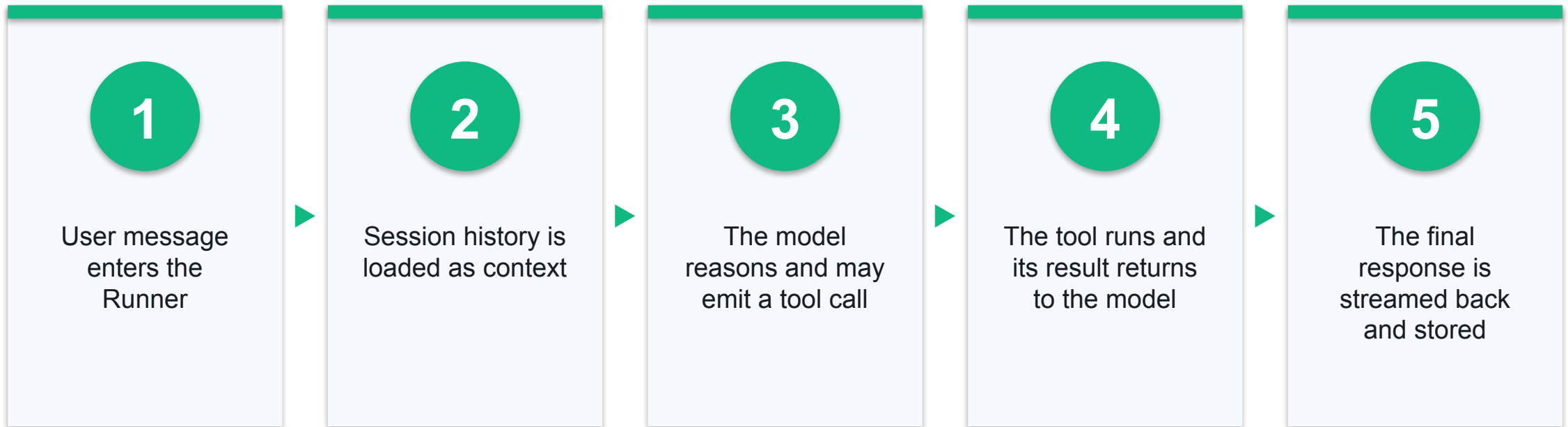
Python functions the agent may call to fetch data or take action.

6

sub_agents

Specialist agents this agent may transfer control to.

The ADK Agent Loop



TOPIC 01

Overview of Agentic AI in Gemini ADK

LLM & agentic AI foundations · Gemini model family · ADK architecture · first agent · tools

Key Concepts — Overview of Agentic AI in Gemini ADK

1

A Large Language Model (LLM) is trained on very large text corpora and generates language autoregressively, one token at a time.

2

Agentic AI adds reasoning, memory, tool use and autonomy on top of an LLM, so the model can act, not just answer.

3

Gemini is Google DeepMind's natively multimodal model family — Ultra, Pro, Flash and Nano — reasoning across text, image, video, audio and code.

4

The Agent Development Kit (ADK) is Google's open-source Python framework for building, evaluating and deploying agents.

Key Concepts — Overview of Agentic AI in Gemini ADK (continued)

1

An ADK Agent is defined by four things: a model, a name, a description and an instruction.

2

Tools are ordinary Python functions the agent may call; the docstring and type hints tell the model when and how to call them.

3

adk web launches a local browser IDE for chatting with an agent and inspecting every event, tool call and trace.

4

LiteLLm lets an ADK agent run on non-Gemini models (OpenAI, Anthropic, Ollama) without changing agent code.

Hands-On Labs — Overview of Agentic AI in Gemini ADK

Labs 1–2

- Set Up the Gemini ADK Environment and Get an API Key
- Build Your First ADK Agent — A Retail Banking Assistant

Labs 3–4

- Give an Agent Tools — Live Weather and Web Search
- Swap the Model — Running an ADK Agent on a Non-Gemini LLM

- —

Set Up the Gemini ADK Environment and Get an API Key

LAB 1

Install the Agent Development Kit toolchain with uv, obtain a free Google AI Studio API key, and store it safely in a .env file so every agent in the course can authenticate.

You'll build

A working Python 3.13 project with google-adk installed and a validated GOOGLE_API_KEY.

Tools: Google AI Studio, uv, Python 3.13, google-adk, python-dotenv

Set Up the Gemini ADK Environment and Get an API Key

Test it

`uv run adk --help` prints the ADK usage banner listing the run, web and eval sub-commands, and no `ModuleNotFoundError` is raised.

Build Your First ADK Agent — A Retail Banking Assistant

LAB 2

Create a single-agent banking customer-service assistant. You learn the four fields that define every ADK agent and see how the instruction alone shapes tone, scope and refusals.

You'll build

basic_agent — a Gemini-powered banking assistant that answers general banking questions and refuses to handle PINs, OTPs or full account numbers.

Tools: google-adk, Gemini 2.0 Flash, adk run, adk web

Build Your First ADK Agent — A Retail Banking Assistant

Test it

The agent answers general banking questions helpfully but declines to disclose or request a PIN, OTP or full account number, and the Events tab shows one LLM call per turn.

Give an Agent Tools — Live Weather and Web Search

LAB 3

Add two Python function tools to an agent: a live OpenWeather lookup and a Tavily web search. You see how the docstring and type hints become the tool contract the model reads.

You'll build

multi_tools_agent — an agent that decides for itself whether a question needs the weather tool, the search tool, both, or neither.

Tools: google-adk, OpenWeather API, Tavily API, Gemini 2.0 Flash

Give an Agent Tools — Live Weather and Web Search

Test it

The weather question produces a `get_weather function_call` with a live temperature; the news question produces a `tavily_search` call; the arithmetic question produces no tool call at all.

Swap the Model — Running an ADK Agent on a Non-Gemini LLM

LAB 4

Use the LiteLlm wrapper to point the same agent at an OpenAI model, then compare it with Gemini on the same prompts to judge quality, latency and cost.

You'll build

agent_model — one agent definition that runs on either Gemini or an OpenAI model by changing a single line.

Tools: google-adk, LiteLlm, Gemini 2.0 Flash, OpenAI GPT-4.1-mini

Swap the Model — Running an ADK Agent on a Non-Gemini LLM

Test it

The same agent runs unchanged on both providers, and you can state which model you would choose for this workload and justify it on quality, latency and cost.

Recap — Overview of Agentic AI in Gemini ADK

- You can now: LO1 / LO2 — establish a working Gemini ADK development environment.
- You can now: LO1 / LO3 — define an agent from a model, name, description and instruction.
- You can now: LO1 / LO3 — extend an agent with custom function tools and evaluate tool selection.
- You can now: LO2 / LO4 — compare models and assess the trade-offs for an engineering process.

TOPIC 02

Build A Multi Agent App with Gemini ADK

Sessions & state · handoff · sub-agents · workflow agents · guardrails · structured output · MCP

Key Concepts — Build A Multi Agent App with Gemini ADK

1

LLMs are stateless — a Session plus a SessionService is what gives an ADK agent memory across turns.

2

The Runner is the execution engine: it takes a user message, drives the agent loop and streams back Events.

3

Multi-agent handoff — a root agent with sub_agents transfers control to whichever specialist matches the request.

4

A hierarchical (coordinator) pattern puts a router agent above specialised sub-agents, each with its own instruction and tools.

5

SequentialAgent runs sub-agents in a fixed order and passes each output forward; ParallelAgent fans them out concurrently.

Key Concepts — Build A Multi Agent App with Gemini ADK (continued)

1

Callbacks such as `before_model_callback` implement guardrails — inspect, block or rewrite a request before it reaches the model.

2

`output_schema` with a Pydantic model forces the agent to emit validated, machine-readable JSON instead of free text.

3

The Model Context Protocol (MCP) is an open standard that lets an agent consume tools hosted by an external server over StreamableHTTP or SSE.

4

Agents can also be declared declaratively in YAML config files instead of Python.

Hands-On Labs — Build A Multi Agent App with Gemini ADK

Labs 5–7

- Give an Agent Memory — Sessions, State and the Runner
- Inspect the Agent Loop — Events, Tool Calls and Final Responses
- Multi-Agent Handoff — Joke Generator to Translator

Labs 8–10

- Hierarchical Multi-Agent System — The Tutor Agent
- Sequential Workflow Agent — Singapore Transport Route Planner
- Add a Guardrail — Blocking Unsafe Requests with a Callback

Labs 11–12

- Structured Output — Forcing Valid JSON with Pydantic
- Connect External Tools with MCP — StreamableHTTP and SSE

Give an Agent Memory — Sessions, State and the Runner

LAB 5

Drive an agent programmatically with a Runner and an InMemorySessionService, so the conversation persists across turns instead of restarting on every message.

You'll build

agent_session — a multi-turn agent that answers follow-up questions using earlier context.

Tools: google-adk, Runner, InMemorySessionService, google.genai types

Give an Agent Memory — Sessions, State and the Runner

Test it

The follow-up 'And what about Osaka?' is understood as a weather question without repeating the word weather — and stops working when the session is removed.

Inspect the Agent Loop — Events, Tool Calls and Final Responses

LAB 6

Stream the Event objects the Runner emits and learn to read an agent trace: which event carries the tool call, which carries the tool result, and which is the final response.

You'll build

agent_interact — an instrumented agent that prints every event in its reasoning loop.

Tools: google-adk, Runner, Events API, InMemorySessionService

Inspect the Agent Loop — Events, Tool Calls and Final Responses

Test it

You can point to the `function_call`, the `function_response` and the final response in the trace, and state the number of LLM round-trips the turn consumed.

Multi-Agent Handoff — Joke Generator to Translator

LAB 7

Build a three-level agent hierarchy where a root agent hands off to a joke generator, which in turn hands off to a translator. This is the core ADK delegation pattern.

You'll build

agent_handoff — a root agent that produces an English joke and its Chinese translation through two automatic handoffs.

Tools: google-adk, sub_agents, Gemini 2.0 Flash

Multi-Agent Handoff — Joke Generator to Translator

Test it

One 'Tell me a joke' request yields an English joke followed by a Chinese translation, and the Events tab shows two `transfer_to_agent` calls.

Hierarchical Multi-Agent System — The Tutor Agent

LAB 8

Build a tutoring system where one root agent routes each question to a maths, physics or history specialist, each with its own instruction and teaching style.

You'll build

tutor_agent — a coordinator with three subject specialists that routes by topic.

Tools: google-adk, sub_agents, coordinator pattern

Hierarchical Multi-Agent System — The Tutor Agent

Test it

Each subject question is answered by the matching specialist, visible as a `transfer_to_agent` event, and your new fourth specialist is routed to correctly.

Sequential Workflow Agent — Singapore Transport Route Planner

LAB 9

Chain three agents in a fixed order — collect the journey, research cross-country options, then produce a full route report — using SequentialAgent with the google_search tool.

You'll build

transport_agent — a sequential pipeline producing a route report by MRT, bus, taxi, cycling and walking.

Tools: google-adk, SequentialAgent, LlmAgent, google_search

Sequential Workflow Agent — Singapore Transport Route Planner

Test it

A single journey request produces one report containing all five transport modes plus a recommended fastest route, with the three stages visible in order in the Events tab.

Add a Guardrail — Blocking Unsafe Requests with a Callback

LAB 10

Use `before_model_callback` to inspect every user message before it reaches the LLM, block requests containing a forbidden keyword, and record the block in session state.

You'll build

agent_guardrail — a tool-using agent that intercepts and refuses blocked requests without ever calling the model.

Tools: `google-adk`, `before_model_callback`, `CallbackContext`, `LlmRequest`, `LlmResponse`

Add a Guardrail — Blocking Unsafe Requests with a Callback

Test it

The blocked keyword returns the refusal message with no model call in the trace, while normal requests still work — and your added keyword is blocked too.

Structured Output — Forcing Valid JSON with Pydantic

LAB 11

Attach a Pydantic `output_schema` so the agent returns a validated Recipe object with typed fields instead of free-form prose that a downstream system cannot parse.

You'll build

agent_structured_output — an agent whose every reply is schema-valid JSON.

Tools: google-adk, Pydantic BaseModel, output_schema

Structured Output — Forcing Valid JSON with Pydantic

Test it

Every response parses as JSON matching the Recipe schema, cooking_time is an integer, and your added difficulty field is populated.

Connect External Tools with MCP — StreamableHTTP and SSE

LAB 12

Connect an ADK agent to a remote MCP server so it can use tools it does not own, using both the StreamableHTTP and SSE transports.

You'll build

agent_mcp and agent_mcp_sse — agents whose toolset is discovered at runtime from an MCP server.

Tools: google-adk, McpToolset, StreamableHTTPConnectionParams, SseConnectionParams, n8n MCP server

Connect External Tools with MCP — StreamableHTTP and SSE

Test it

The agent prints the tools discovered from the MCP server and successfully invokes one; changing the server URL changes the toolset with no code edit.

Recap — Build A Multi Agent App with Gemini ADK

- You can now: LO3 — implement session management so an agent remembers earlier turns.
- You can now: LO3 / LO4 — analyse the agent execution loop and evaluate its behaviour.
- You can now: LO3 — implement agent-to-agent delegation with `sub_agents`.
- You can now: LO3 — design a coordinator agent routing to specialised sub-agents.
- You can now: LO3 / LO2 — orchestrate a fixed multi-stage pipeline with `SequentialAgent`.
- You can now: LO3 / LO4 — implement and evaluate a safety guardrail on an agent.

TOPIC 03

Build Agentic AI RAG in Gemini ADK

RAG pipeline · loading & splitting · embeddings · vector stores · retrieval · grounded answers

Key Concepts — Build Agentic AI RAG in Gemini ADK

1

Retrieval Augmented Generation (RAG) grounds an LLM in your own documents, reducing hallucination and keeping answers current.

2

The RAG pipeline is: load → split → embed → store → retrieve → generate.

3

Loaders convert PDF, HTML, CSV, Word and web sources into a common document object.

4

Splitting breaks documents into chunks small enough to embed and retrieve precisely, with overlap to preserve context.

5

An embedding maps text to a dense vector so that semantically similar text sits close together in vector space.

Key Concepts — Build Agentic AI RAG in Gemini ADK (continued)

1

A vector database (Chroma, FAISS, Pinecone, Milvus) stores embeddings and answers nearest-neighbour similarity queries.

2

Similarity search returns the closest chunks; Maximum Marginal Relevance (MMR) trades a little similarity for diversity.

3

In ADK, retrieval is wrapped as a tool so the agent decides when to consult the knowledge base.

4

RAG performance is evaluated on retrieval quality, groundedness/faithfulness, answer relevance and latency.

Hands-On Labs — Build Agentic AI RAG in Gemini ADK

Labs 13–13

- Load, Split and Embed Documents into a Vector Store

Labs 14–14

- Build the Agentic RAG Agent — Retrieval as a Tool

Labs 15–15

- Evaluate RAG Performance — Retrieval Quality and Groundedness

Load, Split and Embed Documents into a Vector Store

LAB 13

Take two product PDFs, extract their text, split it into retrievable chunks with page metadata, embed them and persist the vectors in a local Chroma database.

You'll build

A persistent Chroma collection holding the chunked and embedded air-fryer product and warranty manuals.

Tools: ChromaDB, pypdf, google-adk, Gemini embeddings

Load, Split and Embed Documents into a Vector Store

Test it

agent_rag/chroma_db exists and contains a populated air_fryer_docs collection, and you can state how the chunking rule changed the number of stored chunks.

Build the Agentic RAG Agent — Retrieval as a Tool

LAB 14

Complete the RAG loop: expose the vector search as an ADK tool, let the agent decide when to retrieve, and require it to cite the source document and page.

You'll build

agent_rag — a grounded question-answering agent over the air-fryer manuals with citations.

Tools: google-adk, ChromaDB, similarity search, Gemini 2.0 Flash

Build the Agentic RAG Agent — Retrieval as a Tool

Test it

Both document questions are answered with the correct source and page cited, the out-of-scope question is refused rather than hallucinated, and you can see the retrieved chunks.

Evaluate RAG Performance — Retrieval Quality and Groundedness

LAB 15

Build a small evaluation set and score your RAG agent on retrieval hit rate, groundedness, answer relevance and latency, then tune one parameter and measure the improvement.

You'll build

A completed RAG evaluation table with a before-and-after comparison for one tuned parameter.

Tools: agent_rag, ChromaDB, similarity search vs MMR

Evaluate RAG Performance — Retrieval Quality and Groundedness

Test it

A completed evaluation table with before-and-after figures for retrieval hit rate, groundedness and latency, plus a written, evidence-based tuning recommendation.

Recap — Build Agentic AI RAG in Gemini ADK

- You can now: LO4 — build the ingestion half of a RAG pipeline and inspect the embeddings.
- You can now: LO4 / LO3 — wrap retrieval as a tool so the agent grounds its answers in your documents.
- You can now: LO4 — evaluate the performance effectiveness of a RAG implementation.

TOPIC 04

Build an Agentic AI App with Gemini Agent ADK and Streamlit

From agent to product · Streamlit chat UI · async Runner · deployment & feasibility

Key Concepts — Build an Agentic AI App with Gemini Agent ADK and Streamlit

1

A production agent app needs a user interface, session handling, error handling and API-key management.

2

Streamlit turns a Python script into a web app, making it the fastest route from an ADK agent to a usable product.

3

`st.session_state` persists the ADK SessionService and the chat history across Streamlit re-runs.

Key Concepts — Build an Agentic AI App with Gemini Agent ADK and Streamlit (continued)

1

The ADK Runner is async, so a Streamlit app drives it through `asyncio.run()` inside the request handler.

2

Secrets belong in `.env` and never in source control or in the repository you push to GitHub.

3

Assessing feasibility means weighing accuracy, latency, token cost, maintainability and governance against the business benefit.

Hands-On Labs — Build an Agentic AI App with Gemini Agent ADK and Streamlit

Labs 16–16

- Declarative Agents — Configuring a Multi-Agent System in YAML

Labs 17–17

- Ship the Agent as a Web App with Streamlit

Labs 18–18

- Capstone — Design, Build and Assess Your Own Multi-Agent Application

Declarative Agents — Configuring a Multi-Agent System in YAML

LAB 16

Define a five-specialist transport assistant entirely in YAML config files, with no Python agent code, and assess what this buys you for maintainability and governance.

You'll build

transport_agent_yaml — a root agent with MRT, bus, taxi, bike and walk specialists, all declared in YAML.

Tools: google-adk YAML config, LlmAgent, Gemini 2.5 Flash

Declarative Agents — Configuring a Multi-Agent System in YAML

Test it

All five specialists route correctly, and your sixth agent works after editing YAML only — no Python file was modified.

Ship the Agent as a Web App with Streamlit

LAB 17

Wrap the sequential transport agent in a Streamlit chat UI, wiring the async ADK Runner and persisting the session across Streamlit re-runs so the conversation survives.

You'll build

transport_agent_streamlit — a browser chat application backed by the multi-agent transport workflow.

Tools: Streamlit, google-adk Runner, InMemorySessionService, asyncio

Ship the Agent as a Web App with Streamlit

Test it

The Streamlit app answers a journey query, retains history across follow-up turns, and your Clear chat button empties the conversation without restarting the server.

Capstone — Design, Build and Assess Your Own Multi-Agent Application

LAB 18

In groups of three to five, design and build a multi-agent ADK application for a use case in your own industry, then present it with a feasibility assessment.

You'll build

A working multi-agent application with at least three agents, at least two tools, session memory and a documented feasibility assessment.

Tools: google-adk, sub_agents or SequentialAgent, custom tools, optional RAG and Streamlit

Capstone — Design, Build and Assess Your Own Multi-Agent Application

Test it

A running application with three or more agents and two or more tools, demonstrated live, plus a feasibility assessment stating whether you would recommend production deployment and on what evidence.

Recap — Build an Agentic AI App with Gemini Agent ADK and Streamlit

- You can now: LO2 / LO3 — separate agent configuration from code to improve maintainability.
- You can now: LO3 — deploy a multi-agent system behind a usable chat interface.
- You can now: LO1 / LO2 / LO3 / LO4 — design a multi-agent solution and assess its feasibility.



WRAP-UP

Course Summary & Next Steps

What You Achieved

1

LO1 · Analysed LLM applications

Explained how LLMs work and identified Generative AI use cases across finance, marketing, education, healthcare and legal.

2

LO2 · Established Gemini designs

Chose Gemini models and tuned parameters, and assessed the improvements agents bring to engineering processes.

3

LO3 · Developed LLM applications

Built single agents, tool-using agents, multi-agent handoff, sequential workflows, guardrails, structured output and MCP integrations.

4

LO4 · Evaluated RAG

Built an agentic RAG pipeline over your own PDFs and measured retrieval hit rate, groundedness and latency.

NEXT STEPS

Continue Your Learning

1

Official ADK docs

<https://google.github.io/adk-docs/> — the reference for every ADK class and pattern.

2

Google AI Studio

<https://aistudio.google.com> — prototype prompts and manage your API keys.

3

Rebuild the labs

Redo each lab from a blank file until the agent pattern is automatic.

4

Extend the capstone

Add RAG, a guardrail and a Streamlit UI to your own multi-agent application.

5

Deploy it

Take your agent to Cloud Run or Vertex AI Agent Engine for production use.

6

Share it

Publish your agent to GitHub and gather feedback from real users.

Recommended Courses

1

WSQ - Develop Generative AI Solutions with Azure OpenAI Service

2

WSQ - Mastering Prompt Engineering for Generative AI Content Creation

3

WSQ - Build LLM Applications Using Langchain and Flowise

4

WSQ - Digital Transformation and Business Innovation with Generative AI (GenAI)

5

WSQ - Practical Image Generation Using GAN, VAE, and Diffusion Models

Support

- If you have any enquiries during or after the class, you can contact us below.
- Email: enquiry@tertiaryinfotech.com
- Tel: +65 6100 0613
- Website: www.tertiarycourses.com.sg
- LMS / TMS: <https://lms-tms.tertiaryinfotech.com>

Assessment

1

Written Assessment (SAQ)

1 hour · open book · short-answer knowledge questions.

2

Practical Performance (PP)

1 hour · open book · hands-on Gemini ADK agent build tasks.

3

Digital attendance

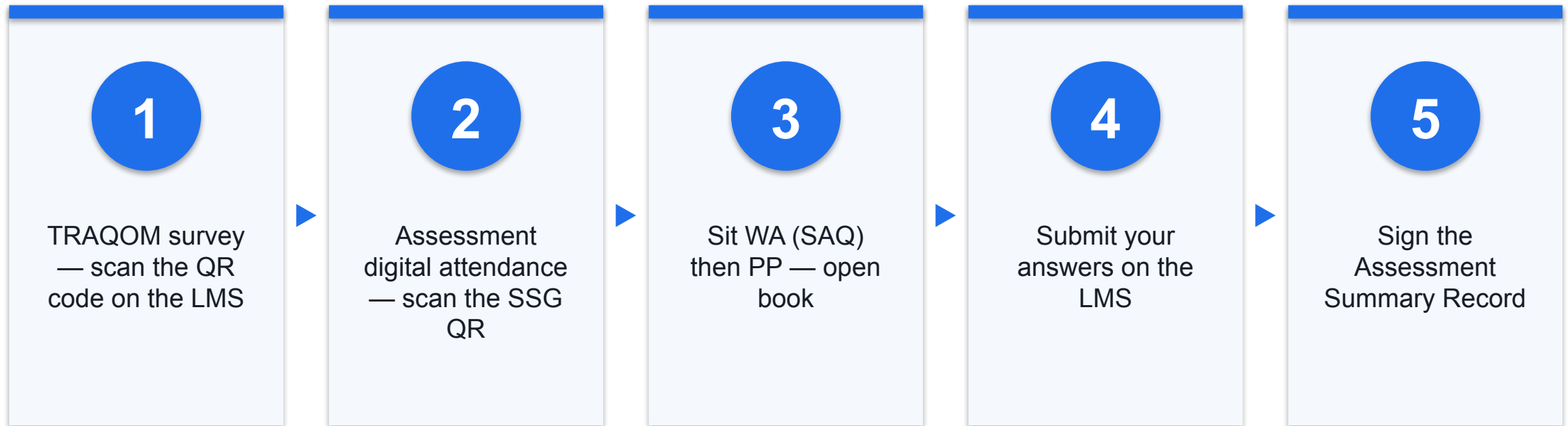
Remember to take the Assessment digital attendance (TRAQOM) before you start.

4

Submit on the LMS

Upload your completed answers at <https://lms-tms.tertiaryinfotech.com/>

Assessment Flow



Digital Attendance (Mandatory)

1

Three times a day

Take the AM, PM and Assessment digital attendance — mandatory for every WSQ-funded course.

2

Trainer shows the QR

The trainer or administrator displays the digital attendance QR code from the SSG portal.

3

Scan and submit

Scan the QR code with your mobile phone camera and submit your attendance.

4

75% minimum

A minimum of 75% attendance is required to be eligible for assessment and funding.

HAPPY BUILDING

Thank You!

You can now design, build and evaluate multi-agent AI applications with the Gemini Agent Development Kit.